

Taller — Gobierno de Arquitectura

Cyber Nexus — Grupo 7 | Universidad de La Sabana — Arquitectura Empresarial

Caso: Plataforma de gestión de propiedades y reservas integrada con Airbnb | 25 de abril de 2026

1. Principios Arquitectónicos

Principio 1 — Seguridad por Diseño

Enunciado: Toda funcionalidad nueva incorpora controles de autenticación, autorización, cifrado y auditoría desde el diseño, no como capa añadida.

Justificación: El Taller 5 identificó tres componentes de riesgo Alto (autenticación, API de reservas, base de datos de clientes). Adoptar este principio evita repetir el patrón de controles ausentes detectado en el Taller 6 y asegura que cualquier nueva funcionalidad considere las amenazas STRIDE aplicables.

Principio 2 — Cumplimiento Normativo No Negociable

Enunciado: Ningún componente entra a producción sin cumplir la Ley 1581/2012, ISO/IEC 27001 y los lineamientos de MinTIC: consentimiento, política de tratamiento, Habeas Data, cifrado, RBAC y logs.

Justificación: El Taller 6 evidenció nivel de madurez bajo, con cuatro criterios “No cumple” y cinco “Parcial”. Elevar el cumplimiento a principio rector evita sanciones legales, protege a los titulares de los datos y construye confianza con clientes y socios externos como Airbnb.

Principio 3 — Alta Disponibilidad y Observabilidad

Enunciado: Ningún componente crítico puede ser un punto único de falla; toda capa expone métricas, logs estructurados y trazas accesibles desde una plataforma centralizada.

Justificación: El Taller 4 identificó SPOFs en Load Balancer, WAF y PostgreSQL Primary, y monitoreo solo en la capa de datos. Este principio cierra simultáneamente la brecha de disponibilidad y la de auditoría detectadas en el Taller 6 y mitiga las amenazas de Repudiation y Denial of Service del Taller 5.

2. Estándares

Los estándares definen las tecnologías y prácticas concretas que aterrizan los principios. Su objetivo es reducir la heterogeneidad técnica, facilitar el mantenimiento y agilizar el onboarding del equipo.

Categoría	Estándar adoptado
Lenguaje / framework	Backend: Node.js (TypeScript) sobre NestJS. Frontend: React + TypeScript.
Comunicación síncrona	REST sobre HTTPS (TLS 1.2+) documentado con OpenAPI 3.0.
Comunicación asíncrona	Eventos vía cola de mensajes AWS SQS/SNS, con dead-letter queue.
Autenticación / autorización	OAuth 2.0 + OIDC con JWT firmado, MFA para administradores y RBAC con roles definidos.
Cifrado	TLS 1.2+ en tránsito; AES-256 en reposo (RDS y S3) gestionado con AWS KMS.
Infraestructura	AWS Cloud Multi-AZ; contenedores en EKS/ECS Fargate; Terraform + GitHub Actions para IaC y CI/CD.
Datos	PostgreSQL en RDS Multi-AZ + Redis para caché; backups automáticos y point-in-time recovery.
Observabilidad	CloudWatch + Prometheus/Grafana; logs JSON con correlation ID; alertas proactivas.

3. Decisiones Arquitectónicas (ADR)

Se documentan dos decisiones arquitectónicas relevantes siguiendo el formato ADR: problema, decisión, justificación, alternativas consideradas y consecuencias.

ADR-001 — RDS PostgreSQL Multi-AZ con failover automático

Problema: El Taller 4 identificó que toda la escritura pasa por un único nodo PostgreSQL Primary y que la promoción manual de la réplica genera ventanas de inactividad. ¿Cómo eliminar este SPOF sin modificar la lógica de los microservicios?

Decisión: Migrar a AWS RDS for PostgreSQL en configuración Multi-AZ con failover automático y mantener una réplica adicional para reportes. Los microservicios apuntan al endpoint gestionado de RDS, que abstrae la conmutación.

Justificación: Failover en menos de dos minutos (elimina el SPOF), backups y point-in-time recovery (criterios 6 y 8 del Taller 6), cifrado en reposo nativo con KMS (criterio 3) y operación gestionada que reduce la carga del equipo.

Alternativas consideradas: Mantener la configuración actual con promoción manual (descartada por el riesgo operativo); Patroni sobre EC2 (mayor control pero exige experiencia operativa que el equipo no tiene); Aurora PostgreSQL (mejor disponibilidad pero costo significativamente mayor).

Consecuencias: (+) Elimina SPOF y avanza tres criterios de cumplimiento. (–) Costo mensual ~1,8x por instancia standby y mayor acoplamiento a AWS. (=) Requiere ventana de mantenimiento para migración y ajustes en monitoreo y alertas.

ADR-002 — Comunicación asíncrona basada en eventos para integración con Airbnb

Problema: La integración actual con Airbnb es síncrona, sin reintento robusto. El Taller 5 identificó amenazas de Tampering e Information Disclosure y el Taller 6 marcó el criterio 7 como Parcial. ¿Cómo procesar los eventos de Airbnb de forma confiable, validada y auditable?

Decisión: Los webhooks se reciben en el Integration Service, se validan (firma HMAC y esquema) y se publican en colas SQS por tipo de evento. Los consumidores (Booking, Property) procesan los eventos de forma independiente, con reintentos automáticos y dead-letter queue para fallos persistentes.

Justificación: Desacopla el ritmo de Airbnb del sistema interno, habilita reintentos con backoff exponencial, cierra Tampering vía HMAC y aporta trazabilidad mediante la DLQ. Permite escalado independiente de cada consumidor.

Alternativas consideradas: Mantener integración síncrona con reintentos básicos (insuficiente ante caídas prolongadas); Apache Kafka gestionado en MSK (capacidad superior pero complejidad operativa innecesaria para el volumen actual); EventBridge (válido pero SQS ofrece control más granular sobre reintentos y DLQ para este caso).

Consecuencias: (+) Mayor resiliencia, mitigación directa de dos amenazas STRIDE y avance en el criterio 7 del Taller 6. (–) Mayor complejidad operativa y consistencia eventual que exige idempotencia en los consumidores. (=) Requiere monitoreo específico de la cola y formación del equipo en patrones de mensajería asíncrona.